

PyGPLA: A Python Toolbox for Generalized Phase Locking Analysis

Amir Khani^{1,2,¶}, Christian Beste^{3,4}, and Shervin Safavi^{2,3,¶}

¹ Donders Institute for Brain, Cognition, and Behaviour, Radboud University, Nijmegen, Netherlands ² Department of Computer Science, Amirkabir University of Technology, Tehran, Iran ³ Cognitive Neurophysiology, Department of Child and Adolescent Psychiatry, Faculty of Medicine, Technische Universität Dresden, Dresden, Germany ⁴ German Center for Child and Adolescent Health (DZKJ), partner site Leipzig/Dresden, Dresden, Germany ⁵ Computational Neuroscience, Department of Child and Adolescent Psychiatry, Faculty of Medicine, Technische Universität Dresden, Dresden 01307, Germany ⁶ Department of Computational Neuroscience, Max Planck Institute for Biological Cybernetics, Tübingen 72076, Germany ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

PyGPLA is a Python implementation of Generalized Phase Locking Analysis (GPLA) for multivariate analysis of coupling between spikes and local field potentials (LFPs) (Safavi et al., 2023). For a given frequency, GPLA constructs a complex coupling matrix $\hat{C}(f) \in \mathbb{C}^{N_c \times N_u}$ between LFP channels (N_c) and spike units (N_u), then applies singular value decomposition (SVD) to reduce the dimensionality of data. The leading singular value summarizes population-level coupling strength, while the corresponding singular vectors describe dominant LFP and spike coupling modes. PyGPLA accepts a user-provided frequency-specific analytic LFP signal or phase representation and provides data selection, optional whitening and normalization, coupling-matrix construction, SVD-based decomposition, and statistical significance testing (Safavi et al., 2021).

Statement of need

Neural recordings are becoming increasingly high-dimensional and multimodal, demanding more sophisticated analysis tools. Simultaneous analysis of spiking activity and LFPs is among the most informative multi-modal approaches in systems neuroscience, providing insight into the multi-scale mechanisms underlying cognitive functions such as attention and memory (Buzsaki et al., 2012; Einevoll et al., 2013; Herreras, 2016). LFP oscillatory activity partly reflects subthreshold processes shared by neuronal ensembles, and the synchronization between this activity and spiking is hypothesized to coordinate neural populations during cognitive processes (Buzsaki et al., 2012; Hagen et al., 2016).

However, commonly used spike-LFP coupling measures are pairwise, making them suboptimal for modern multichannel recordings (Jiang et al., 2015; Li et al., 2016; Vinck et al., 2010, 2012; Zarei et al., 2018; Zeitler et al., 2006). As modern electrophysiological techniques allow simultaneous recordings from hundreds or even thousands of sites, pairwise analyses generate high-dimensional covariance matrices whose size grows rapidly with the number of channels, limiting interpretable extraction of large-scale collective dynamics (Buzsaki, 2004; Dickey et al., 2009; Juavinett et al., 2019; Jun et al., 2017).

GPLA was developed to address these challenges by providing an efficient multivariate framework together with statistical routines (Safavi et al., 2021) for characterizing spike-LFP coupling at the population level (Safavi et al., 2023). However, the original algorithm was implemented in MATLAB, limiting its accessibility. PyGPLA bridges this gap as an open-source Python implementation, Aligning with the extensive use of Python in neuroscience (Akam et al., 2022;

Behrad et al., 2025; Freeman, 2015; Goodman & Brette, 2008; Gramfort et al., 2013, 2013; Ince et al., 2009; Krause & Lindemann, 2014; Makowski et al., 2021; Muller et al., 2015; Nouri et al., 2025; Peirce, 2009; Tennøe et al., 2018; Viejo et al., 2023; Zeraati et al., 2022).

Existing spike-field coupling methods - including the phase-locking value (PLV), pairwise phase consistency (PPC) (Vinck et al., 2010), and spike-field coherence - operate on individual spike-LFP channel pairs. While informative for small-scale recordings, these pairwise approaches do not directly capture the population-level structure of coupling, which is of paramount importance for recordings with modern high-density probes. GPLA addresses this gap by providing a multivariate method that summarizes all spike-field interactions simultaneously, analogous to how multivariate statistical methods such as principal component analysis (PCA) summarize covariance structure. To our knowledge, no other Python package implements this multivariate approach to spike-field coupling analysis.

State of the field

Common spike-field coupling measures, including the phase-locking value, pairwise phase consistency, and spike-field coherence, quantify relationships between individual spike units and LFP channels (Vinck et al., 2010, 2012). General-purpose Python packages such as MNE-Python provide extensive signal-processing and electrophysiology infrastructure (Gramfort et al., 2013), but do not implement GPLA's joint decomposition of the complete spike-field coupling matrix. The only directly comparable implementation is the MATLAB research code accompanying the original GPLA study (Safavi et al., 2023). PyGPLA was developed as a separate package because GPLA requires a method-specific pipeline comprising coupling-matrix construction, normalization, optional reduced-rank whitening, SVD-based extraction of population coupling modes, and analytical or surrogate-based statistical inference. These operations do not extend an existing pairwise measure or general preprocessing function in a natural way. PyGPLA's scholarly contribution is therefore a dedicated, open-source Python implementation of the complete GPLA methodology.

Functionality

Core analysis pipeline

PyGPLA provides a comprehensive solution for multivariate spike-field coupling analysis, including:

- **LFP input and data preparation:** PyGPLA accepts a user-provided complex analytic LFP signal or a real phase representation in radians. Raw LFP voltage must first be converted upstream to the desired frequency-specific analytic signal, for example through band-pass filtering followed by a Hilbert transform (Chavez et al., 2006). PyGPLA then supports temporal and unit selection, optional channel-wise normalization, and optional reduced-rank whitening to decorrelate channels while avoiding noise amplification (Safavi et al., 2023).
- **Coupling-matrix construction:** Assembly of the complex-valued coupling matrix $\hat{C}(f) \in \mathbb{C}^{N_c \times N_u}$, where each entry sums the analytic LFP evaluated at all spike times of a given unit.
- **SVD-based decomposition:** Extraction of the generalized phase locking value or gPLV (the leading singular value) and associated LFP and spike spatial vectors, with rotational phase alignment, unwhitening of LFP vectors, and spike-vector rescaling.
- **Statistical testing:** Significance assessment via two complementary approaches: (1) surrogate-based testing using multiple spike-jittering schemes, and (2) an analytical test based on Marchenko–Pastur Random Matrix Theory (RMT) (Anderson et al., 2010; Safavi et al., 2023).
- **Simulation tools:** Synthetic data generators for phase-locked and transient-coupling scenarios, supporting reproducibility and method validation.

93 Normalization options

94 The package supports two normalization modes for the coupling matrix, expressed in terms
95 of the total spike count N_m^{tot} for unit m : PLV-type normalization ($1/N_m^{\text{tot}}$) for phase-only
96 analysis, and unit-variance normalization ($1/\sqrt{N_m^{\text{tot}}}$) required for the theoretical RMT-based
97 significance test.

98 Example

99 We illustrate PyGPLA on synthetic transient-coupling simulations generated with the script
100 `paper/figures/figure2.py`. As shown in Figure 1, the figure compares four coupling models
101 and reports their recovered gPLV values, together with representative LFP and spike-train
102 windows and the associated spike-vector structure.

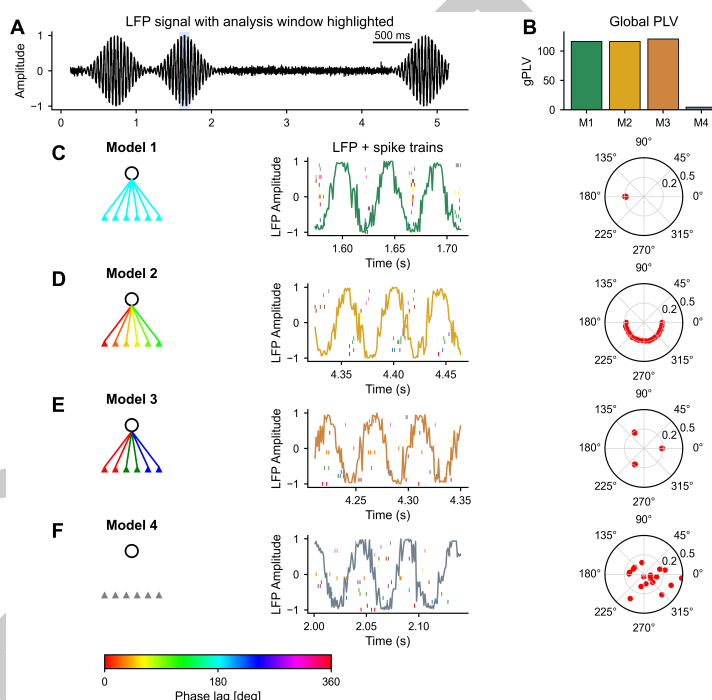


Figure 1: Illustrative PyGPLA simulation. (A) LFP analysis window. (B) gPLV across Models 1-4. (C-F) Model schematics, LFP and spikes, and recovered spike vectors.

103 Code snippets and detailed instructions for reproducing these results are available in the
104 package documentation and example scripts in the repository.

105 Implementation details

106 The `pygpla` package is distributed under the BSD-2-Clause license. PyGPLA follows a
107 layered architecture separating preprocessing (`pygpla.preprocessing`), core computation
108 (`pygpla.core`), statistical testing (`pygpla.stats`), simulation utilities (`pygpla.simulations`),
109 and configuration (`pygpla.config`). The high-level API (`pygpla.api.gpla`) orchestrates the
110 full pipeline in a single call, keeping default usage compact while preserving access to lower-level
111 components for advanced analyses. The package is built on NumPy (Harris et al., 2020) and
112 SciPy (Virtanen et al., 2020), and includes a pytest test suite with continuous integration via
113 GitHub Actions.

114 Use of generative AI

115 Generative AI tools were used only to assist with drafting documentation and refactoring
 116 code for style. All AI-assisted output was reviewed, tested, and validated by the authors, who
 117 take full responsibility for the correctness of the software and the content of this paper. The
 118 scientific method, algorithmic design, and numerical implementation of GPLA were carried out
 119 by the authors.

120 Acknowledgments

121 We acknowledge contributions from collaborators who provided feedback during the
 122 development of pyGPLA. S.S. acknowledges support from the Max Planck Society and an
 123 add-on fellowship from the Joachim Herz Foundation. C.B. acknowledges support from the
 124 Federal Ministry of Research, Technology and Space (Bundesministerium für Forschung,
 125 Technologie und Raumfahrt; BMFTR) as part of the German Center for Child and Adolescent
 126 Health (DZKJ) under funding code 01GL2405B.

127 References

- 128 Akam, T., Lustig, A., Rowland, J. M., Kapaniaiah, S. K. T., Esteve-Agraz, J., Panniello, M.,
 129 Márquez, C., Kohl, M. M., Kätzel, D., Costa, R. M., & Walton, M. E. (2022). Open-source,
 130 python-based, hardware and software for controlling behavioural neuroscience experiments.
 131 *eLife*, 11, e67846. <https://doi.org/10.7554/eLife.67846>
- 132 Anderson, G. W., Guionnet, A., & Zeitouni, O. (2010). *An introduction to random matrices*.
 133 Cambridge University Press.
- 134 Behrad, A., Ostrow, M., Fakharian, M. T., Fiete, I., Beste, C., & Safavi, S. (2025). Fast
 135 dynamical similarity analysis. *arXiv*, 2511.22828, 1–31. <https://doi.org/10.48550/arXiv.2511.22828>
- 136 Buzsaki, G. (2004). Large-scale recording of neuronal ensembles. *Nature Neuroscience*, 7(5),
 137 446–451. <https://doi.org/10.1038/nrn1233>
- 138 Buzsaki, G., Anastassiou, C. A., & Koch, C. (2012). The origin of extracellular fields and
 139 currents—EEG, ECoG, LFP and spikes. *Nature Reviews Neuroscience*, 13(6), 407–420.
 140 <https://doi.org/10.1038/nrn3241>
- 141 Chavez, M., Besserve, M., Adam, C., & Martinerie, J. (2006). Towards a proper estimation
 142 of phase synchronization from time series. *Journal of Neuroscience Methods*, 154(1-2),
 143 149–160. <https://doi.org/10.1016/j.jneumeth.2005.12.009>
- 144 Dickey, A. S., Suminski, A. J., Amit, Y., & Hatsopoulos, N. G. (2009). Single-unit stability
 145 using chronically implanted multielectrode arrays. *Journal of Neurophysiology*, 102(2),
 146 1331–1339. <https://doi.org/10.1152/jn.90920.2008>
- 147 Einevoll, G. T., Kayser, C., Logothetis, N. K., & Panzeri, S. (2013). Modelling and analysis
 148 of local field potentials for studying the function of cortical circuits. *Nature Reviews*
 149 *Neuroscience*, 14(11), 770–785. <https://doi.org/10.1038/nrn3599>
- 150 Freeman, J. (2015). Open source tools for large-scale neuroscience. *Current Opinion in*
 151 *Neurobiology*, 32, 156–163. <https://doi.org/10.1016/j.conb.2015.04.002>
- 152 Goodman, D. F. M., & Brette, R. (2008). Brian: A simulator for spiking neural networks in
 153 python. *Frontiers in Neuroinformatics*, 2, 5. <https://doi.org/10.3389/neuro.11.005.2008>
- 154 Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C.,
 155 Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. S. (2013). MEG and
 156 EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7(267), 1–13. <https://doi.org/10.3389/fnins.2013.00267>
- 157
 158

- Hagen, E., Dahmen, D., Stavrinou, M. L., Linden, H., Tetzlaff, T., Albada, S. J. van, & Diesmann, M. (2016). Hybrid scheme for modeling local field potentials from point-neuron networks. *Cerebral Cortex*, 26(12), 4461–4496. <https://doi.org/10.1093/cercor/bhw237>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Herreras, O. (2016). Local field potentials: Myths and misunderstandings. *Frontiers in Neural Circuits*, 10, 101. <https://doi.org/10.3389/fncir.2016.00101>
- Ince, R. A. A., Petersen, R. S., Swan, D. C., & Panzeri, S. (2009). Python for information theoretic analysis of neural data. *Frontiers in Neuroinformatics*, 3, 4. <https://doi.org/10.3389/neuro.11.004.2009>
- Jiang, H., Bahramisharif, A., Gerven, M. A. J. van, & Jensen, O. (2015). Measuring directionality between neuronal oscillations of different frequencies. *NeuroImage*, 118, 359–367. <https://doi.org/10.1016/j.neuroimage.2015.05.044>
- Juavinett, A. L., Bekheet, G., & Churchland, A. K. (2019). Chronically implanted neuropixels probes enable high-yield recordings in freely moving mice. *eLife*, 8, e47188. <https://doi.org/10.7554/eLife.47188>
- Jun, J. J., Steinmetz, N. A., Siegle, J. H., Denman, D. J., Bauza, M., Barbarits, B., & others. (2017). Fully integrated silicon probes for high-density recording of neural activity. *Nature*, 551, 232–236. <https://doi.org/10.1038/nature24636>
- Krause, F., & Lindemann, O. (2014). Expyriment: A python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, 46, 416–428. <https://doi.org/10.3758/s13428-013-0390-6>
- Li, Z., Cui, D., & Li, X. (2016). Unbiased and robust quantification of synchronization between spikes and local field potential. *Journal of Neuroscience Methods*, 269, 33–38. <https://doi.org/10.1016/j.jneumeth.2016.05.004>
- Makowski, D., Pham, T., Lau, Z. J., Brammer, J. C., Lespinasse, F., Pham, H., Schölzel, C., & Chen, S. H. A. (2021). NeuroKit2: A python toolbox for neurophysiological signal processing. *Behavior Research Methods*, 53, 1689–1696. <https://doi.org/10.3758/s13428-020-01516-y>
- Muller, E., Bednar, J. A., Diesmann, M., Gewaltig, M.-O., Hines, M., & Davison, A. P. (2015). Python in neuroscience. *Frontiers in Neuroinformatics*, 9, 11. <https://doi.org/10.3389/fninf.2015.00011>
- Nouri, S., Shao, K., & Safavi, S. (2025). TranCIT: Transient causal interaction toolbox. *Journal of Open Source Software*, 10(116), 9302. <https://doi.org/10.21105/joss.09302>
- Peirce, J. W. (2009). Generating stimuli for neuroscience using PsychoPy. *Frontiers in Neuroinformatics*, 2, 10. <https://doi.org/10.3389/neuro.11.010.2008>
- Safavi, S., Logothetis, N. K., & Besserve, M. (2021). From univariate to multivariate coupling between continuous signals and point processes: A mathematical framework. *Neural Computation*, 33(7), 1751–1817.
- Safavi, S., Panagiotaropoulos, T. I., Kapoor, V., Ramirez-Villegas, J. F., Logothetis, N. K., & Besserve, M. (2023). Uncovering the organization of neural circuits with generalized phase locking analysis. *PLoS Computational Biology*, 19(4), e1010983. <https://doi.org/10.1371/journal.pcbi.1010983>
- Tennøe, S., Halnes, G., & Einevoll, G. T. (2018). Uncertainpy: A python toolbox for uncertainty quantification and sensitivity analysis in computational neuroscience. *Frontiers in Neuroinformatics*, 12, 49. <https://doi.org/10.3389/fninf.2018.00049>
- Viejo, G., Levenstein, D., Skromne Carrasco, S., Mehrotra, D., Mahallati, S., Vite, G. R.,

- 206 Denny, H., Sjulson, L., Battaglia, F. P., & Peyrache, A. (2023). Pynapple, a toolbox for
207 data analysis in neuroscience. *eLife*, 12, RP85786. <https://doi.org/10.7554/eLife.85786.3>
- 208 Vinck, M., Battaglia, F. P., Womelsdorf, T., & Pennartz, C. M. A. (2012). Improved measures
209 of phase-coupling between spikes and the local field potential. *Journal of Computational*
210 *Neuroscience*, 33(1), 53–75. <https://doi.org/10.1007/s10827-011-0374-4>
- 211 Vinck, M., Wingerden, M. van, Womelsdorf, T., Fries, P., & Pennartz, C. M. A. (2010). The
212 pairwise phase consistency: A bias-free measure of rhythmic neuronal synchronization.
213 *NeuroImage*, 51(1), 112–122. <https://doi.org/10.1016/j.neuroimage.2010.01.073>
- 214 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
215 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
216 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17, 261–272.
217 <https://doi.org/10.1038/s41592-019-0686-2>
- 218 Zarei, M., Jahed, M., & Daliri, M. R. (2018). Introducing a comprehensive framework to
219 measure spike-LFP coupling. *Frontiers in Computational Neuroscience*, 12, 78. <https://doi.org/10.3389/fncom.2018.00078>
- 220
221 Zeitler, M., Fries, P., & Gielen, S. (2006). Assessing neuronal coherence with single-unit,
222 multi-unit, and local field potentials. *Neural Computation*, 18, 2256–2281. <https://doi.org/10.1162/neco.2006.18.9.2256>
- 223
224 Zeraati, R., Engel, T. A., & Levina, A. (2022). A flexible bayesian framework for unbiased
225 estimation of timescales. *Nature Computational Science*, 2, 193–204. <https://doi.org/10.1038/s43588-022-00214-3>
- 226

DRAFT